The Visual Interface

Systems overview

CateringPro Services needs a digital solution that allows corporate clients to place catering orders online, customise what they want, and then track the order status from preparation through to delivery. The system will use a login to identify the user, then provide an ordering interface that collects client details, event details, and selected menu items. When an order is submitted, the system will store the data across the correct tables (`tbl_client`, `tbl_order`, `tbl_order_item`, and related lookup tables such as `tbl_menu_item` and `tbl_manager` if required). The solution must support staff viewing and updating orders so that clients receive accurate progress updates, which links directly to the client brief's requirement for streamlined ordering and real-time tracking.

Inputs identified

These are the inputs that the user (client) and system (staff/database) will provide.

Login inputs

- Email or username
- Password

Order creation inputs (client-facing)

- Client first name
- Client surname
- Client address

- Client postcode
- Event location
- Order date
- Menu item selection (one or more items)
- Quantity for each menu item

System and staff inputs (internal)

- Assigned account manager (selected by staff or auto-assigned)
- Order status update (e.g. Preparing, Confirmed, Delivered)
- Total cost (calculated by the system or entered by staff in a prototype)

Process description

The algorithm design will be shown using one main flowchart that focuses on login, then calls subroutines to handle data entry and database inserts.

Main process (Login flowchart)

1. Display login form.
2. User enters username/email and password.
3. Presence check both fields.
4. Compare credentials to stored user records.
5. If credentials are valid, create a logged-in session and load the ordering page.
6. If credentials are invalid, show an error message and allow retry.

Subroutine: Add or confirm client record

1. Collect client details from the order form.
2. Validate that required fields are present.
3. Check if the client already exists (for example using postcode and surname, or an email if your design includes it).
4. If the client exists, retrieve `client_id`.
5. If the client does not exist, insert the client and return the new `client_id`.

Subroutine: Create a new order

1. Use `client_id` from the client subroutine.
2. Collect order details (order date, event location).
3. Set default order status (e.g. “Preparing” for a new order).
4. Assign or select an account manager.
5. Insert into `tbl_order` and return the new `order_id`.

Subroutine: Add order items

1. For each selected menu item:
    a. Read menu item choice and quantity.
    b. Validate that quantity is a positive whole number.
    c. Insert a row into `tbl_order_item` using `order_id` and `menu_item_id`.
2. Repeat until all menu items are inserted.

Subroutine: Handle errors and user messages

1. If validation fails, do not insert data.
2. Display a clear error message explaining what must be corrected.
3. Keep user-entered data where possible to reduce frustration.

Outputs identified

These are the outputs produced by the system for both clients and staff.

Client outputs

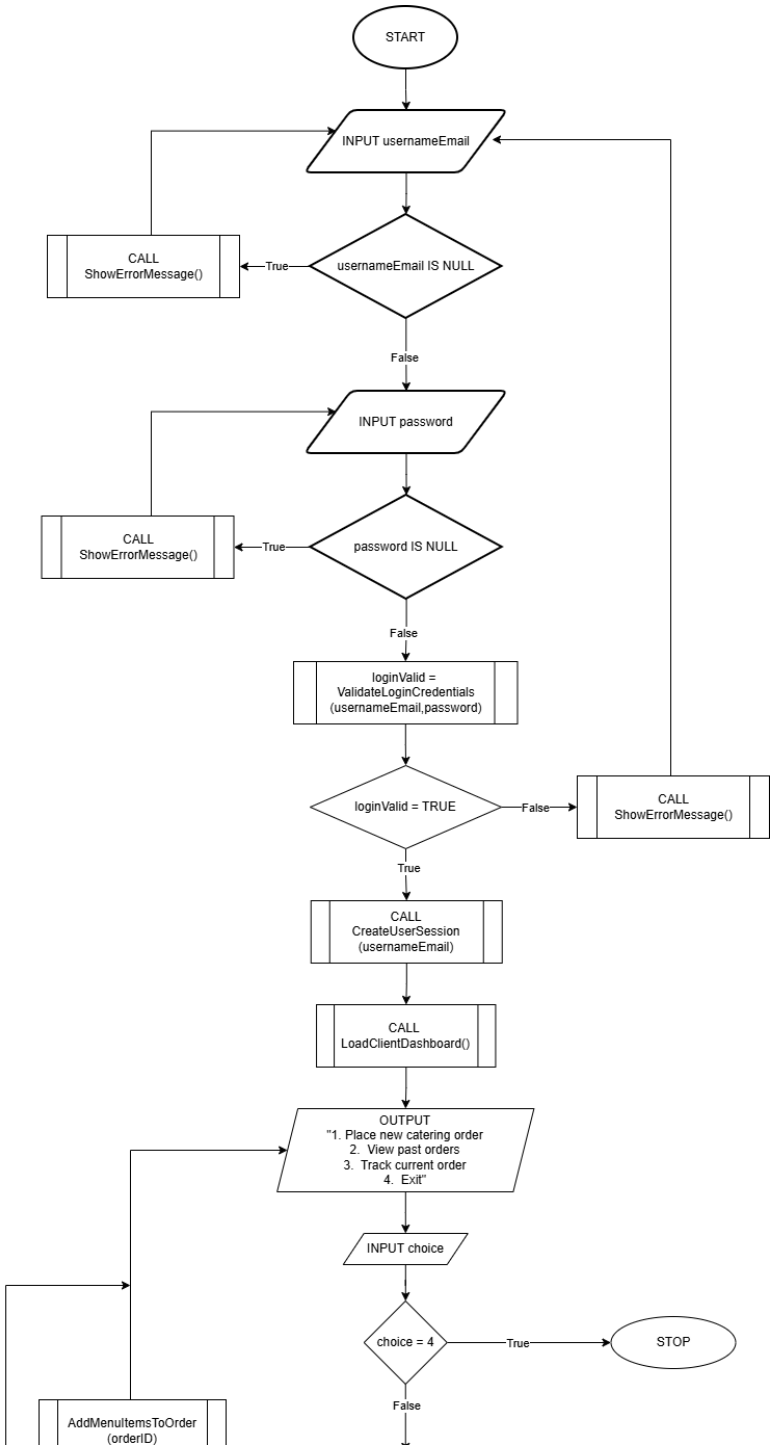
- Login success or login error message
- Confirmation that the order has been submitted
- A generated order reference (OrderID)
- A summary of the order (event location, date, selected menu items, quantities)
- Real-time order tracking information (current order status)

Staff outputs

- List of orders filtered by status (e.g. show all “Preparing”)
- Full order breakdown including menu items and quantities
- Client contact and delivery details for fulfilment
- Ability to update status and confirm progress

Algorithm design

Flowchart: Client Login and Access to CateringPro Ordering Portal


START
INPUT usernameEmail
usernameEmail IS NULL
True
CALL
ShowErrorMessage()
False
INPUT password
password IS NULL
True
CALL
ShowErrorMessage()
False
loginValid =
ValidateLoginCredentials
(usernameEmail,password)
loginValid = TRUE
False
CALL
ShowErrorMessage()
True
CALL
CreateUserSession
(usernameEmail)
CALL
LoadClientDashboard()
OUTPUT
"1. Place new catering order
2. View past orders
3. Track current order
4. Exit"
INPUT choice
choice = 4
True
STOP
False
AddMenuItemsToOrder
(orderID)

This flowchart represents the secure entry point to the CateringPro system. Corporate clients must log in before they can place catering orders, view previous orders, or track progress from preparation to delivery. Login ensures each order is linked to the correct client account, which supports streamlined ordering, accurate communication, and improved customer satisfaction.

Pseudocode: Subroutine to Validate Login Credentials

```
FUNCTION ValidateLoginCredentials(usernameEmail, password)
BEGIN FUNCTION

    SET loginValid TO FALSE

    # Retrieve password hash for the given email from Tbl_User
    SET storedPasswordHash TO SELECT password_hash FROM Tbl_User WHERE email =
usernameEmail

    # Log the login attempt for audit purposes
    WRITE Tbl_LoginAudit usernameEmail, CURRENT_TIME, "Attempt"

    IF storedPasswordHash <> "" THEN
        IF VerifyPassword(password, storedPasswordHash) = TRUE THEN
            SET loginValid TO TRUE

            # Log successful login
            WRITE Tbl_LoginAudit usernameEmail, CURRENT_TIME, "Success"
        ELSE
            # Log failed password verification
            WRITE Tbl_LoginAudit usernameEmail, CURRENT_TIME, "FailedPassword"
        END IF
    ELSE
        # Log failed email lookup
```

```
        WRITE Tbl_LoginAudit usernameEmail, CURRENT_TIME, "FailedEmail"
    END IF

    RETURN loginValid

END FUNCTION
```

This subprogram is called in the flowchart at the point where the system needs to decide whether the user is allowed into the CateringPro portal. It supports the scenario by ensuring only authorised corporate clients can access the online ordering system and real-time tracking features. In a full solution, this would check a user table in the database rather than using a placeholder comparison.

Pseudocode: Subroutine to Create User Session

```
FUNCTION CreateUserSession(usernameEmail)
BEGIN FUNCTION

    DECLARE sessionLoggedIn
    DECLARE sessionUser
    DECLARE sessionStartTime
    DECLARE sessionExpiryTime

    SET sessionUser TO usernameEmail
    SET sessionLoggedIn TO TRUE
    SET sessionStartTime TO CURRENT_TIME
    SET sessionExpiryTime TO sessionStartTime + 2 HOURS

    # Log session creation for audit
    SEND "Session created for: " & sessionUser & " at " & sessionStartTime TO DISPLAY

    # Store session details in session table
```

```
    WRITE Tbl_Session sessionUser, sessionStartTime, sessionExpiryTime, sessionLoggedIn

    RETURN sessionLoggedIn

END FUNCTION
```

This subprogram is called after successful login in the flowchart. It represents the step where the system keeps the client logged in so they can place an order, view previous orders, or track order progress without logging in repeatedly. This links to the scenario by supporting a smooth user experience for corporate clients and improving customer engagement.

Pseudocode: Subroutine to Load Client Dashboard

```
FUNCTION LoadClientDashboard()
BEGIN FUNCTION

    SEND "Dashboard loaded" TO DISPLAY
    SEND "Welcome, corporate client!" TO DISPLAY

    # Display main dashboard options
    SEND "Options:" TO DISPLAY
    SEND "1. Place new order" TO DISPLAY
    SEND "2. View past orders" TO DISPLAY
    SEND "3. Track current orders" TO DISPLAY
    SEND "4. Update account details" TO DISPLAY
    SEND "5. Contact support" TO DISPLAY

    # Show summary of recent activity
    SEND "Recent orders:" TO DISPLAY
    # Assume GetRecentOrders() returns a list of recent orders
    FOR EACH order FROM GetRecentOrders(usernameEmail) DO
        SEND order TO DISPLAY
    END FOREACH
```

```
    # Show tracking status for active orders
    SEND "Active order tracking:" TO DISPLAY
    FOR EACH activeOrder FROM GetActiveOrders(usernameEmail) DO
        SEND "Order ID: " & activeOrder.OrderID & " Status: " & activeOrder.Status TO
DISPLAY
    END FOREACH

END FUNCTION
```

This is called once login is successful. In the flowchart it represents moving the user into the main portal area. It links to the scenario because the dashboard is where corporate clients access the key features from the brief, including placing custom catering orders and checking order tracking updates.

PSEUDOCODE: Subroutine to Show Error Message

```
FUNCTION ShowErrorMessage(errorType)
BEGIN FUNCTION

    IF errorType = "Missing username/email" THEN
        SEND "Enter your username/email" TO DISPLAY
    ELSE IF errorType = "Missing password" THEN
        SEND "Enter your password" TO DISPLAY
    ELSE IF errorType = "Invalid login" THEN
        SEND "Login failed. Check your details and try again." TO DISPLAY
    ELSE
        SEND "An error occurred. Please contact support if the problem continues." TO
DISPLAY
    END IF

    # Log the error for audit and support
    WRITE Tbl_ErrorLog errorType, CURRENT_TIME
```

```
END FUNCTION
```

This subprogram is called at multiple points in the flowchart when something goes wrong with login input or credential checking. Clear error messages reduce user frustration and make the ordering portal easier to use. This supports the scenario objective of improving customer satisfaction by keeping the system professional and straightforward.

Pseudocode: Subroutine to Add or Retrieve Client Record

```
FUNCTION AddOrRetrieveClientRecord(clientFirstName, clientSurname, clientAddress,
clientPostcode)
BEGIN FUNCTION

    DECLARE clientID
    DECLARE clientExists

    SET clientExists TO FALSE
    SET clientID TO 0

    # Lookup client in tbl_client by surname and postcode
    SET clientID TO SELECT client_id FROM tbl_client WHERE surname = clientSurname AND
postcode = clientPostcode

    IF clientID <> 0 THEN
        SET clientExists TO TRUE
    END IF

    IF clientExists = TRUE THEN
        # Log retrieval for audit
        WRITE Tbl_ClientAudit clientID, CURRENT_TIME, "Retrieved"
        RETURN clientID
    ELSE
```

```
        # Insert new client record into tbl_client
        SET clientID TO INSERT INTO tbl_client VALUES (clientFirstName, clientSurname,
clientAddress, clientPostcode)
        # Log creation for audit
        WRITE Tbl_ClientAudit clientID, CURRENT_TIME, "Created"
        RETURN clientID
    END IF

END FUNCTION
```

This is called after the client chooses to place a new order in the flowchart. It ensures the system has a valid `clientID` to link orders to the correct corporate client. This supports the scenario by streamlining the ordering process and preventing duplicated client details, which improves record keeping and future engagement such as loyalty and repeat business.

Pseudocode: Subroutine to Create a New Catering Order

```
FUNCTION CreateNewCateringOrder(clientID, orderDate, eventLocation)
BEGIN FUNCTION

    DECLARE orderID
    DECLARE managerID
    DECLARE orderStatus
    DECLARE totalCost
    DECLARE orderCreatedTime

    SET managerID TO 1
    SET orderStatus TO "Preparing"
    SET totalCost TO 0
    SET orderCreatedTime TO CURRENT_TIME

    # Insert new order record into tbl_order
```

```
    SET orderID TO INSERT INTO tbl_order VALUES (clientID, orderDate, eventLocation,
managerID, orderStatus, totalCost, orderCreatedTime)

    # Log order creation for audit
    WRITE Tbl_OrderAudit orderID, clientID, orderCreatedTime, "Created"

    RETURN orderID

END FUNCTION
```

This subprogram is called immediately after a client is identified. It creates the main order record in the database and returns an `orderID` so that menu items can be added in a linked table. This supports the scenario because the online ordering system must store each order properly and assign an initial tracking status, which enables real-time progress updates later.

Pseudocode: Subroutine to Add Menu Items to an Order

```
FUNCTION AddMenuItemsToOrder(orderID)
BEGIN FUNCTION

    DECLARE menuItemID
    DECLARE quantity
    DECLARE addAnother

    SET addAnother TO "Y"

    WHILE addAnother = "Y" DO
        # Prompt for menu item selection
        SEND "Enter menu item ID:" TO DISPLAY
        RECEIVE menuItemID FROM (INTEGER) KEYBOARD

        SEND "Enter quantity:" TO DISPLAY
        RECEIVE quantity FROM (INTEGER) KEYBOARD
```

```
        # Insert menu item into tbl_order_item
        INSERT INTO tbl_order_item VALUES (orderID, menuItemID, quantity)

        SEND "Item added to order" TO DISPLAY

        SEND "Add another item? (Y/N):" TO DISPLAY
        RECEIVE addAnother FROM (CHARACTER) KEYBOARD
    END WHILE

    SEND "Order items complete" TO DISPLAY

    # Log order item entry for audit
    WRITE Tbl_OrderItemAudit orderID, CURRENT_TIME, "ItemsAdded"

END FUNCTION
```

This subprogram is called after an order is created in the flowchart. It allows the corporate client to customise their catering order by adding multiple menu items with quantities. This directly supports the scenario requirement for a user-friendly ordering system where clients can tailor catering packages, and it creates the detailed order breakdown needed for preparation, delivery, and tracking.

Pseudocode: Subprogram to complete necessary Validation checks

```
FUNCTION ValidateEntry(inputValue)
BEGIN FUNCTION

    DECLARE isValid
    DECLARE errorMessage

    SET isValid TO TRUE
```

```
    SET errorMessage TO ""

    # Presence check
    IF inputValue = "" THEN
        SET isValid TO FALSE
        SET errorMessage TO "Input cannot be empty."
    END IF

    # Format check (example: email format)
    IF isValid = TRUE THEN
        IF NOT (inputValue CONTAINS "@" AND inputValue CONTAINS ".") THEN
            SET isValid TO FALSE
            SET errorMessage TO "Input must be a valid email address."
        END IF
    END IF

    # Type check (example: input should be INTEGER)
    IF isValid = TRUE THEN
        IF NOT (inputValue = inputValue DIV 1) THEN
            SET isValid TO FALSE
            SET errorMessage TO "Input must be a whole number."
        END IF
    END IF

    # Range check (example: input between 1 and 100)
    IF isValid = TRUE THEN
        IF inputValue < 1 OR inputValue > 100 THEN
            SET isValid TO FALSE
            SET errorMessage TO "Input must be between 1 and 100."
        END IF
    END IF

    IF isValid = TRUE THEN
        RETURN "Valid"
```

```
    ELSE
        RETURN errorMessage
    END IF

END FUNCTION
```

This subprogram is called whenever the system needs to validate user input, such as during registration, login, or order placement. It demonstrates how presence, format, type, and range checks can be combined to ensure data integrity and user-friendly feedback. By providing clear error messages for each validation failure, the system helps users correct mistakes quickly, supporting the scenario objective of a professional, reliable, and easy-to-use portal for corporate clients. This approach improves data quality, reduces input errors, and enhances overall customer satisfaction

Variable List

Variable Name	Data Type	Description	Used In
usernameEmail	STRING	Stores login email entered by client	Main Flowchart
password	STRING	Stores login password entered by client	Main Flowchart
loginValid	BOOLEAN	Indicates if login credentials are valid	ValidateLoginCredentials
clientID	INTEGER	Unique identifier for client	AddOrRetrieveClientRecord
orderID	INTEGER	Unique identifier for order	CreateNewCateringOrder
managerID	INTEGER	Identifies assigned account manager	CreateNewCateringOrder
orderDate	DATE	Date order is placed	CreateNewCateringOrder
eventLocation	STRING	Location of catering event	CreateNewCateringOrder
orderStatus	STRING	Current stage of order	CreateNewCateringOrder
totalCost	DECIMAL	Total cost of catering order	CreateNewCateringOrder
menuItemID	INTEGER	Identifies selected catering item	AddMenuItemsToOrder
quantity	INTEGER	Number of items requested	AddMenuItemsToOrder
addAnother	STRING	Controls loop for multiple items	AddMenuItemsToOrder
errorType	STRING	Determines error message shown	ShowErrorMessage

The variable list defines the key pieces of data used within the algorithm to support the CateringPro ordering system. Input variables such as `usernameEmail`, `password`, `eventLocation`, `menuItemID`, and `quantity` capture the information entered by corporate clients through the website interface. Processing variables such as `clientID`, `orderID`, `managerID`, and `loginValid` are used to control logic, maintain relationships between tables, and ensure correct database inserts. Output variables such as `errorType` and confirmation messages provide structured feedback to the user. Clearly defining variable names, data types, and their purpose ensures logical consistency, supports accurate SQL operations in Supabase, and aligns the algorithm directly with the scenario requirement for secure login, customised ordering, and real-time tracking.

Error Handling Plan

Error Type	Where Occurs	System Action	User Feedback
Missing username/email	Login	Stop validation	“Enter your username/email”
Missing password	Login	Stop validation	“Enter your password”
Invalid credentials	Login	Deny access	“Login failed. Try again.”
Missing event location	Order creation	Prevent insert	“Event location required”
Quantity less than 1	AddMenuItemsToOrder	Prevent insert	“Quantity must be 1 or more”
Foreign key constraint failure	tbl_order insert	Cancel transaction	“Order could not be processed”
Network/Supabase error	Any DB call	Retry option	“Connection error. Please try again.”

The error handling plan ensures that the CateringPro system behaves predictably and professionally when incorrect or incomplete data is entered or when database operations fail. Validation checks prevent missing login credentials, invalid quantities, and incomplete order details from being processed. Database-level constraints such as foreign key checks and quantity limits protect data integrity within Supabase. Handling network or connection errors ensures that corporate clients are informed if an order cannot be processed immediately. By identifying these potential failures in advance and defining clear system responses, the design supports the client brief objective of streamlining the ordering process, improving customer satisfaction, and maintaining accurate real-time order tracking.

Section Conclusion

This algorithm design provides a clear end to end structure for the CateringPro digital solution, starting with user inputs at login and moving through the key steps needed to create an order that can be tracked and managed. The flowchart sets out the main control logic, while the linked pseudocode subprograms show how the system will validate access, identify the correct client, create an order record, and store multiple menu items and quantities in a reliable way. This directly supports the scenario objectives by streamlining the ordering process for corporate clients, improving customer experience through clear feedback, and laying the groundwork for real time order tracking from preparation to delivery.

During the design phase, there are a few additional elements worth considering to make the solution more robust and realistic. User accounts should be stored in a dedicated table with securely hashed passwords rather than placeholder checks. The system should include clear business rules for order statuses, including who is allowed to update them, to ensure tracking is accurate. It may also be useful to include pricing per menu item so total cost can be calculated automatically rather than entered manually, which would reduce human error. Finally, the design should consider accessibility and usability from the start, including clear labels, keyboard navigation, readable error messages, and a simple order review step before submission so corporate clients can confirm details before committing the order.

Data Requirements

Normalisation

The dataset provided represents CateringPro's current order records stored in a single, unstructured table. It combines information about orders, clients, account managers, event locations and menu items all in one place. This structure contains non-atomic values and repeating groups, which makes it inefficient and prone to duplication and update errors. To improve the design, we will apply the normalisation process step by step. First, we will convert the data from Unnormalised Form (UNF) to First Normal Form (1NF) by ensuring all fields contain atomic values. Next, we will move to Second Normal Form (2NF) by removing partial dependencies and separating data into appropriate entity tables. Finally, we will achieve Third

Normal Form (3NF) by eliminating transitive dependencies so that each table stores data about one thing only. This process will result in a well-structured relational database that reduces redundancy and improves data integrity.

UNF

Order ID	Client Name	Order Date	Account Manager ID	Order Status	Menu Items Ordered	Event Location	Client ID	Manager First Name	Client Address	Client Postcode	Total Cost
1	Sarah Mitchell	10/03/2025	3	Preparing	Sandwich Platters (5), Fruit Trays (2)	London Office	1	Pauline	352 Cornwall Avenue	QJ2 7YN	£850
2	Daniel Gilbert	12/03/2025	1	Delivered	Hot Buffet (1), Drinks Package (3), Dessert Selection (2)	Manchester HQ	2	Mariya	126 Vane Avenue	MW6 6KZ	£1,450
3	Chloe Fields	14/03/2025	3	Confirmed	Breakfast Package (2), Coffee Service (1)	London Office	3	Pauline	314 Clifton Street	BV5 1OA	£620
4	Sarah Mitchell	18/03/2025	2	Preparing	Canapés (100), Champagne Service (5)	Birmingham Branch	1	Aranos	352 Cornwall Avenue	QJ2 7YN	£2,200
5	Thomas Ferguson	20/03/2025	3	Confirmed	Cold Buffet (3), Soft	Leeds Centre	4	Pauline	94 Deans Avenue	FQ4 2PH	£980

					Drinks (10)						
6	Emma Wallace	22/03/2025	1	Delivered	Hot Buffet (2), Dessert Selection (4), Coffee Service (3)	Manchester HQ	5	Mariya	142 Viaduct Street	DA08SD	£1,750
7	Ryan Spencer	25/03/2025	3	Preparing	Vegan Platters (6), Fruit Trays (3)	Glasgow Office	6	Pauline	75 Broughton Row	AE5 9QA	£900
8	Sofia Medina	27/03/2025	2	Confirmed	Breakfast Package (4), Pastries (20)	Birmingham Branch	7	Aranos	112 Hillview	DV06QF	£1,200
9	Liam Cooper	29/03/2025	1	Delivered	Premium Buffet (2), Wine Selection (6), Dessert Selection (3)	Manchester HQ	8	Mariya	114 Albert Avenue	MF35MC	£2,800

This dataset is in **Unnormalised Form (UNF)** because it contains non-atomic data and repeating groups within single fields. For example, the **Client Name** column stores both first name and surname together, which means it does not contain a single atomic value. The **Menu Items Ordered** column contains multiple products and quantities in one cell, creating a repeating group inside a single attribute. In addition, the table mixes multiple entities such as clients, managers, orders and event

locations into one structure, leading to redundancy and data duplication. These issues mean the table does not meet the requirements of First Normal Form.

1NF

Table 1: Tbl_Order (1NF)

Primary Key: OrderID

OrderID (PK)	Order Date	ClientID	ClientFirstName	ClientSurname	ClientAddress	ClientPostcode	EventLocation	ManagerID	ManagerFirstName	OrderStatus	Total Cost
1	10/03/2025	1	Sarah	Mitchell	352 Cornwall Avenue	QJ2 7YN	London Office	3	Pauline	Preparing	£850
2	12/03/2025	2	Daniel	Gilbert	126 Vane Avenue	MW6 6KZ	Manchester HQ	1	Mariya	Delivered	£1,450
3	14/03/2025	3	Chloe	Fields	314 Clifton Street	BV5 1OA	London Office	3	Pauline	Confirmed	£620
4	18/03/2025	1	Sarah	Mitchell	352 Cornwall Avenue	QJ2 7YN	Birmingham Branch	2	Aranos	Preparing	£2,200
5	20/03/2025	4	Thomas	Ferguson	94 Deans Avenue	FQ4 2PH	Leeds Centre	3	Pauline	Confirmed	£980
6	22/03/2025	5	Emma	Wallace	142 Viaduct Street	DA0 8SD	Manchester HQ	1	Mariya	Delivered	£1,750

7	25/03/2025	6	Ryan	Spencer	75 Broughton Row	AE5 9QA	Glasgow Office	3	Pauline	Preparing	£900
8	27/03/2025	7	Sofia	Medina	112 Hillview	DV0 6QF	Birmingham Branch	2	Aranos	Confirmed	£1,200
9	29/03/2025	8	Liam	Cooper	114 Albert Avenue	MF3 5MC	Manchester HQ	1	Mariya	Delivered	£2,800

Table 2: Tbl_OrderItem (1NF)

Composite Primary Key: OrderID + MenuItemName
Foreign Key: OrderID references Tbl_Order(OrderID)

OrderID (FK)	MenuItemName	Quantity
1	Sandwich Platters	5
1	Fruit Trays	2
2	Hot Buffet	1
2	Drinks Package	3
2	Dessert Selection	2
3	Breakfast Package	2
3	Coffee Service	1
4	Canapés	100
4	Champagne Service	5
5	Cold Buffet	3
5	Soft Drinks	10
6	Hot Buffet	2

6	Dessert Selection	4
6	Coffee Service	3
7	Vegan Platters	6
7	Fruit Trays	3
8	Breakfast Package	4
8	Pastries	20
9	Premium Buffet	2
9	Wine Selection	6
9	Dessert Selection	3

To convert the dataset into First Normal Form (1NF), the non-atomic fields were separated and the repeating group was removed. The original "Client Name" field was split into separate `ClientFirstName` and `ClientSurname` fields so that each column contains a single value. The "Menu Items Ordered" column, which previously stored multiple items in one cell, was separated into a second table so that each menu item now appears in its own row with its quantity. This ensures that all values are atomic, there are no repeating groups, and each record can be uniquely identified.

2NF

Table 1: Tbl_Client

Primary Key: ClientID

ClientID (PK)	ClientFirstName	ClientSurname	ClientAddress	ClientPostcode
1	Sarah	Mitchell	352 Cornwall Avenue	QJ2 7YN
2	Daniel	Gilbert	126 Vane Avenue	MW6 6KZ
3	Chloe	Fields	314 Clifton Street	BV5 1OA
4	Thomas	Ferguson	94 Deans Avenue	FQ4 2PH
5	Emma	Wallace	142 Viaduct Street	DA0 8SD

6	Ryan	Spencer	75 Broughton Row	AE5 9QA
7	Sofia	Medina	112 Hillview	DV0 6QF
8	Liam	Cooper	114 Albert Avenue	MF3 5MC

Table 2: Tbl_Manager

Primary Key: ManagerID

ManagerID (PK)	ManagerFirstName
1	Mariya
2	Aranos
3	Pauline

Table 3: Tbl_Order

Primary Key: OrderID
Foreign Keys: ClientID, ManagerID

OrderID (PK)	OrderDate	EventLocation	OrderStatus	TotalCost	ClientID (FK)	ManagerID (FK)
1	10/03/2025	London Office	Preparing	£850	1	3
2	12/03/2025	Manchester HQ	Delivered	£1,450	2	1
3	14/03/2025	London Office	Confirmed	£620	3	3
4	18/03/2025	Birmingham Branch	Preparing	£2,200	1	2
5	20/03/2025	Leeds Centre	Confirmed	£980	4	3
6	22/03/2025	Manchester HQ	Delivered	£1,750	5	1
7	25/03/2025	Glasgow Office	Preparing	£900	6	3

8	27/03/2025	Birmingham Branch	Confirmed	£1,200	7	2
9	29/03/2025	Manchester HQ	Delivered	£2,800	8	1

Table 4: Tbl_OrderItem

Composite Primary Key: OrderID + MenuItemName

Foreign Key: OrderID references Tbl_Order(OrderID)

OrderID (FK)	MenuItemName	Quantity
1	Sandwich Platters	5
1	Fruit Trays	2
2	Hot Buffet	1
2	Drinks Package	3
2	Dessert Selection	2
3	Breakfast Package	2
3	Coffee Service	1
4	Canapés	100
4	Champagne Service	5
5	Cold Buffet	3
5	Soft Drinks	10
6	Hot Buffet	2
6	Dessert Selection	4
6	Coffee Service	3
7	Vegan Platters	6
7	Fruit Trays	3
8	Breakfast Package	4
8	Pastries	20

9	Premium Buffet	2
9	Wine Selection	6
9	Dessert Selection	3

This design is in Second Normal Form (2NF) because it is already in 1NF and all non-key attributes depend on the whole key in their table. The repeating client and manager details were removed from the order table and placed into separate tables (`Tbl_Client` and `Tbl_Manager`) so that client attributes depend only on `ClientID` and manager attributes depend only on `ManagerID`. The `Tbl_Order` table now contains only order-specific data plus foreign keys, and `Tbl_OrderItem` contains only item-specific data linked to an order. This removes partial dependency problems and reduces duplication across the database.

3NF

Table 1: Tbl_Client (3NF)

Primary Key: ClientID

ClientID (PK)	ClientFirstName	ClientSurname	ClientAddress	ClientPostcode
1	Sarah	Mitchell	352 Cornwall Avenue	QJ2 7YN
2	Daniel	Gilbert	126 Vane Avenue	MW6 6KZ
3	Chloe	Fields	314 Clifton Street	BV5 1OA
4	Thomas	Ferguson	94 Deans Avenue	FQ4 2PH
5	Emma	Wallace	142 Viaduct Street	DA0 8SD
6	Ryan	Spencer	75 Broughton Row	AE5 9QA
7	Sofia	Medina	112 Hillview	DV0 6QF
8	Liam	Cooper	114 Albert Avenue	MF3 5MC

Table 2: Tbl_Manager (3NF)

Primary Key: ManagerID

ManagerID (PK)	ManagerFirstName
1	Mariya
2	Aranos
3	Pauline

Table 3: Tbl_Order (3NF)

Primary Key: OrderID
Foreign Keys: ClientID, ManagerID

OrderID (PK)	OrderDate	EventLocation	OrderStatus	TotalCost	ClientID (FK)	ManagerID (FK)
1	10/03/2025	London Office	Preparing	£850	1	3
2	12/03/2025	Manchester HQ	Delivered	£1,450	2	1
3	14/03/2025	London Office	Confirmed	£620	3	3
4	18/03/2025	Birmingham Branch	Preparing	£2,200	1	2
5	20/03/2025	Leeds Centre	Confirmed	£980	4	3
6	22/03/2025	Manchester HQ	Delivered	£1,750	5	1
7	25/03/2025	Glasgow Office	Preparing	£900	6	3
8	27/03/2025	Birmingham Branch	Confirmed	£1,200	7	2
9	29/03/2025	Manchester HQ	Delivered	£2,800	8	1

Table 4: Tbl_MenuItem (3NF)

Primary Key: MenuItemID

MenuItemID (PK)	MenuItemName
1	Sandwich Platters
2	Fruit Trays
3	Hot Buffet
4	Drinks Package
5	Dessert Selection
6	Breakfast Package
7	Coffee Service
8	Canapés
9	Champagne Service
10	Cold Buffet
11	Soft Drinks
12	Vegan Platters
13	Pastries
14	Premium Buffet
15	Wine Selection

Table 5: Tbl_OrderItem (3NF Link Table)

Primary Key: OrderItemID

Foreign Keys: OrderID, MenuItemID

OrderItemID (PK)	OrderID (FK)	MenuItemID (FK)	Quantity
1	1	1	5
2	1	2	2
3	2	3	1
4	2	4	3
5	2	5	2

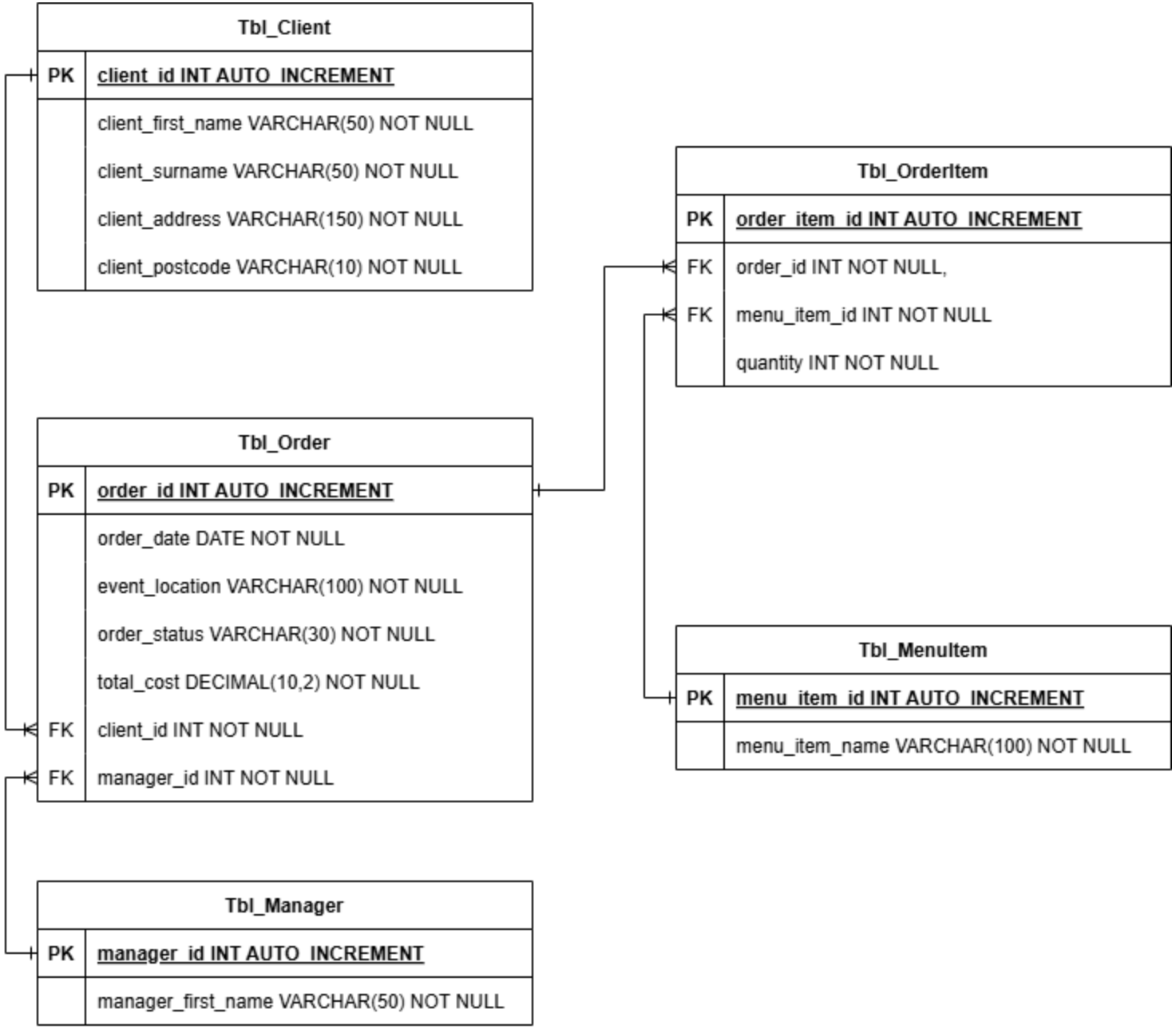
6	3	6	2
7	3	7	1
8	4	8	100
9	4	9	5
10	5	10	3
11	5	11	10
12	6	3	2
13	6	5	4
14	6	7	3
15	7	12	6
16	7	2	3
17	8	6	4
18	8	13	20
19	9	14	2
20	9	15	6
21	9	5	3

This design is in Third Normal Form (3NF) because it is already in 2NF and all non-key attributes depend only on the primary key within each table, with no transitive dependencies. To reach 3NF, the menu item name was removed from the order items table and stored once in a separate `Tbl_MenuItem` table. The link table (`Tbl_OrderItem`) now uses only integer keys to connect orders to menu items, which prevents duplicated item descriptions and ensures consistent reporting and data integrity.

The normalisation process has transformed the original unstructured CateringPro order data into a structured relational database that directly supports the client's objectives. By progressing from UNF to 3NF, repeating groups such as multiple menu items per order were separated into a link table, and duplicated client and manager details were stored once in dedicated tables. This ensures that corporate client information, order details and catering selections are managed accurately and consistently. Removing redundancy reduces the risk of data inconsistencies when updating client details,

tracking order status, or calculating total costs. This structured approach provides the stable backend foundation required to streamline the online ordering process, support accurate real-time order tracking, and enable future development of features such as loyalty and rewards tracking, as outlined in the client brief.

ERD (Entity relationship diagram)


Tbl_Client
PK client_id INT AUTO_INCREMENT
client_first_name VARCHAR(50) NOT NULL
client_surname VARCHAR(50) NOT NULL
client_address VARCHAR(150) NOT NULL
client_postcode VARCHAR(10) NOT NULL
Tbl_OrderItem
PK order_item_id INT AUTO_INCREMENT
FK order_id INT NOT NULL,
FK menu_item_id INT NOT NULL
quantity INT NOT NULL
Tbl_Order
PK order_id INT AUTO_INCREMENT
order_date DATE NOT NULL
event_location VARCHAR(100) NOT NULL
order_status VARCHAR(30) NOT NULL
total_cost DECIMAL(10,2) NOT NULL
FK client_id INT NOT NULL
FK manager_id INT NOT NULL
Tbl_MenuItem
PK menu_item_id INT AUTO_INCREMENT
menu_item_name VARCHAR(100) NOT NULL
Tbl_Manager
PK manager_id INT AUTO_INCREMENT
manager_first_name VARCHAR(50) NOT NULL

The Entity Relationship Diagram clearly demonstrates how the system supports CateringPro's digital solution by linking clients, managers, orders and menu items through defined primary and foreign key relationships. The one-to-many relationship between clients and orders reflects how corporate customers may place multiple catering requests, while the link table between orders and menu items enables customised catering packages — a key requirement of the online ordering system. The defined relationships ensure that order progress can be tracked accurately from creation through preparation to delivery, supporting the real-time tracking objective. By enforcing referential integrity and clear data relationships, the ERD shows how the database structure underpins efficient order management, improved customer engagement, and scalable system development aligned to CateringPro's corporate catering services.

Data Dictionary

Tbl_Client

Field Name	Data Type	Size / Format	Primary Key	Foreign Key	Null Allowed	Example Value	Description	Validation Rules
client_id	INT	Auto Increment	Yes	No	No	1	Unique identifier for each corporate client placing catering orders.	Must be numeric and unique. System generated.
client_first_name	VARCHAR	50	No	No	No	Sarah	The client contact's first name.	Must not be blank. Max 50 characters.
client_surname	VARCHAR	50	No	No	No	Mitchell	The client contact's surname.	Must not be blank. Max 50 characters.

client_address	VARCHAR	150	No	No	No	352 Cornwall Avenue	The billing or registered address for the client organisation/contact.	Must not be blank. Max 150 characters.
client_postcode	VARCHAR	10	No	No	No	QJ2 7YN	Postcode for the client address.	Must not be blank. Max 10 characters. Postcode format expected.

Tbl_Manager

Field Name	Data Type	Size / Format	Primary Key	Foreign Key	Null Allowed	Example Value	Description	Validation Rules
manager_id	INT	Auto Increment	Yes	No	No	3	Unique identifier for each CateringPro account manager handling corporate orders.	Must be numeric and unique. System generated.
manager_first_name	VARCHAR	50	No	No	No	Pauline	First name of the account manager assigned to the order.	Must not be blank. Max 50 characters.

Tbl_MenuItem

Field Name	Data Type	Size / Format	Primary Key	Foreign Key	Null Allowed	Example Value	Description	Validation Rules
menu_item_id	INT	Auto Increment	Yes	No	No	5	Unique identifier for each menu item offered by CateringPro.	Must be numeric and unique. System generated.
menu_item_name	VARCHAR	100	No	No	No	Dessert Selection	Name of a catering item/package available for orders.	Must not be blank. Max 100 characters. Should be unique to avoid duplicates.

Tbl_Order

Field Name	Data Type	Size / Format	Primary Key	Foreign Key	Null Allowed	Example Value	Description	Validation Rules
order_id	INT	Auto Increment	Yes	No	No	9	Unique identifier for each catering order placed by a corporate client.	Must be numeric and unique. System generated.
order_date	DATE	YYYY-MM-DD	No	No	No	2025-03-29	Date the order was placed or confirmed.	Must be a valid date. Cannot be blank.
event_location	VARCHAR	100	No	No	No	Manchester HQ	Delivery/event location for	Must not be blank. Max

							the catering order.	100 characters.
order_status	VARCHAR	30	No	No	No	Delivered	Current stage of the order (used for tracking).	Must not be blank. Should match agreed status values.
total_cost	DECIMAL	10,2	No	No	No	2800.00	Total cost of the order for billing and reporting.	Must be numeric. Must be 0 or higher. 2 d.p.
client_id	INT	Integer	No	Yes (Tbl_Client)	No	8	Links the order to the corporate client who placed it.	Must exist in Tbl_Client. Cannot be blank.
manager_id	INT	Integer	No	Yes (Tbl_Manager)	No	1	Links the order to the account manager responsible.	Must exist in Tbl_Manager. Cannot be blank.

Tbl_OrderItem (Link Table)

Field Name	Data Type	Size / Format	Primary Key	Foreign Key	Null Allowed	Example Value	Description	Validation Rules
order_item_id	INT	Auto Increment	Yes	No	No	21	Unique identifier for each menu	Must be numeric and unique.

							item line within an order.	System generated.
order_id	INT	Integer	No	Yes (Tbl_Order)	No	9	Links a line item back to the order it belongs to.	Must exist in Tbl_Order. Cannot be blank.
menu_item_id	INT	Integer	No	Yes (Tbl_MenuItem)	No	5	Links a line item to the menu item being ordered.	Must exist in Tbl_MenuItem. Cannot be blank.
quantity	INT	Whole number	No	No	No	3	Quantity of the menu item requested within the order.	Must be numeric. Must be 1 or higher.

This data dictionary supports CateringPro's digital solution by clearly defining how corporate client orders are stored, linked, and managed in a reliable database structure. The `Tbl_Order` and `Tbl_OrderItem` tables enable clients to customise catering packages by selecting multiple menu items and quantities per order, which directly supports the online ordering system requirement. The `order_status` field provides a foundation for real-time order tracking so clients can see progress from preparation to delivery. Storing corporate client and account manager details in their own tables ensures consistent communication, accurate billing, and improved customer engagement. Overall, this structure is scalable and well suited to CateringPro's goal of streamlining ordering while improving satisfaction through clear updates, organised data, and future-ready features such as loyalty and rewards tracking.

SQL

Data Types Used

- INT – Used for all primary and foreign keys (client_id, order_id, manager_id, menu_item_id)
- VARCHAR – Used for names, locations and status fields

- DATE – Used for order_date
- DECIMAL (10,2) – Used for total_cost
- BOOLEAN – Used in login validation logic
- INTEGER – Used for quantity

Data Structures Used

- Relational tables to store structured order data
- Link table (tbl_order_item) to manage many-to-many relationship between orders and menu items
- Loop structure (WHILE) in pseudocode to handle multiple menu items per order

The selected data types and structures directly support the CateringPro online ordering system and its implementation in Supabase using SQL. Integer data types are used for all primary and foreign keys to ensure efficient indexing and reliable table relationships between clients, orders and menu items. VARCHAR fields store names, locations and status values to allow flexible but controlled text input. The DATE type ensures accurate recording of catering event dates, which is essential for scheduling and tracking. DECIMAL is used for total_cost to guarantee accurate financial calculations, and INTEGER is used for quantity to enforce whole number ordering. The relational table structure, supported by a link table to manage the many-to-many relationship between orders and menu items, ensures that customised catering packages can be stored efficiently and queried reliably. This structured design provides a stable foundation for SQL operations within Supabase and supports streamlined ordering, accurate tracking and consistent data management as required by the client brief.

TABLE 1: tbl_client

```
CREATE TABLE IF NOT EXISTS tbl_client (
  client_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  client_first_name VARCHAR(50) NOT NULL,
  client_surname VARCHAR(50) NOT NULL,
```

```
  client_address VARCHAR(150) NOT NULL,
  client_postcode VARCHAR(10) NOT NULL
);

INSERT INTO tbl_client (client_first_name, client_surname, client_address, client_postcode)
VALUES
('Sarah', 'Mitchell', '352 Cornwall Avenue', 'QJ2 7YN'),
('Daniel', 'Gilbert', '126 Vane Avenue', 'MW6 6KZ');

SELECT client_id, client_first_name, client_surname, client_postcode
FROM tbl_client
WHERE client_postcode = 'QJ2 7YN';
```

This table stores corporate client information required for placing catering orders. Creating the table ensures client data is stored once and referenced through foreign keys in the order system. The inserted records provide sample data so the website can retrieve real client details during development in Visual Studio Code. The SELECT query demonstrates how the system can filter clients, which supports order lookup, account management, and personalised engagement features outlined in the CateringPro brief.

TABLE 2: tbl_manager

```
CREATE TABLE IF NOT EXISTS tbl_manager (
  manager_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  manager_first_name VARCHAR(50) NOT NULL
);

INSERT INTO tbl_manager (manager_first_name)
```

```
VALUES
('Pauline'),
('Mariya');

SELECT manager_id, manager_first_name
FROM tbl_manager
WHERE manager_first_name = 'Pauline';
```

This table stores CateringPro account managers who oversee corporate catering orders. Separating managers into their own table ensures accurate assignment and avoids duplication across orders. The sample inserts allow the website to display which manager is responsible for a client's order. The SELECT query shows how the system can retrieve specific staff information, supporting internal coordination and improved communication, which aligns with the objective of enhancing customer satisfaction.

TABLE 3: tbl_menu_item

```
CREATE TABLE IF NOT EXISTS tbl_menu_item (
  menu_item_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  menu_item_name VARCHAR(100) NOT NULL
);

INSERT INTO tbl_menu_item (menu_item_name)
VALUES
('Hot Buffet'),
('Dessert Selection');

SELECT menu_item_id, menu_item_name
```

```
FROM tbl_menu_item
WHERE menu_item_name = 'Hot Buffet';
```

This table stores the catering packages and menu options offered by CateringPro. By storing menu items separately, the system allows corporate clients to customise orders while maintaining consistency in reporting and pricing. The inserted records allow the website to populate dropdown menus or selection lists within the online ordering system. The SELECT query demonstrates how specific menu items can be retrieved when building interactive ordering features in VSC.

TABLE 4: tbl_order

```
CREATE TABLE IF NOT EXISTS tbl_order (
  order_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_date DATE NOT NULL,
  event_location VARCHAR(100) NOT NULL,
  order_status VARCHAR(30) NOT NULL,
  total_cost NUMERIC(10,2) NOT NULL CHECK (total_cost >= 0),
  client_id BIGINT NOT NULL REFERENCES tbl_client(client_id),
  manager_id BIGINT NOT NULL REFERENCES tbl_manager(manager_id)
);

INSERT INTO tbl_order (order_date, event_location, order_status, total_cost, client_id,
manager_id)
VALUES
('2025-03-10', 'London Office', 'Preparing', 850.00, 1, 1),
('2025-03-12', 'Manchester HQ', 'Delivered', 1450.00, 2, 2);

UPDATE tbl_order
```

```
SET order_status = 'Delivered'
WHERE order_id = 1;

SELECT order_id, order_date, event_location, order_status, total_cost
FROM tbl_order
WHERE order_status = 'Delivered';
```

This table stores each catering order placed by corporate clients. It links directly to clients and managers through foreign keys, ensuring referential integrity and accurate tracking. The inserted data allows the website prototype to display order histories and status updates. The UPDATE statement demonstrates how CateringPro staff would change order status to support real-time tracking. The SELECT query demonstrates real-time filtering of orders by status, directly supporting the scenario requirement for live order tracking from preparation through to delivery.

TABLE 5: tbl_order_item (Link Table)

```
CREATE TABLE IF NOT EXISTS tbl_order_item (
  order_item_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id BIGINT NOT NULL REFERENCES tbl_order(order_id),
  menu_item_id BIGINT NOT NULL REFERENCES tbl_menu_item(menu_item_id),
  quantity INT NOT NULL CHECK (quantity >= 1)
);

INSERT INTO tbl_order_item (order_id, menu_item_id, quantity)
VALUES
(1, 1, 1),
(2, 2, 2);
```

```
SELECT order_item_id, order_id, menu_item_id, quantity
FROM tbl_order_item
WHERE quantity >= 2;
```

This link table enables the many-to-many relationship between orders and menu items, allowing corporate clients to customise catering packages by selecting multiple items per order. The table ensures quantities are stored correctly and linked through integer foreign keys. The inserted records allow the website to display detailed order breakdowns. The SELECT query demonstrates how the system can retrieve specific line items, supporting accurate billing, reporting, and customer transparency, all of which contribute to the streamlined ordering process described in the client brief.

Section Conclusion

The SQL implementation provides the technical foundation required to support the CateringPro digital ordering system. The CREATE TABLE statements establish a structured relational database that enforces primary and foreign key relationships, ensuring accurate links between clients, orders and menu items. The INSERT statements demonstrate how data is stored during the ordering process, while the SELECT queries show how information can be retrieved for features such as order tracking and client account management. Together, these SQL operations illustrate how the website built in Visual Studio Code will interact with Supabase to securely store, manage and retrieve catering data. This supports the client brief by enabling streamlined corporate ordering, structured data management and reliable real-time order updates.